A3 – Simulated Annealing and Tabu Search

Group number	Members
4	Gerard Figueras, Natan Viejo, Alan Fijal, Mireia Calviño

1. Code explanation

```python
def load_data( ):
    # Implement a function to load all data except the first row and first column in your code.
    # We strongly encourage you to store your data in a NumPy array.

    data = np.genfromtxt("SP500_data.csv", delimiter=",", skip_header=1) # Load the data from the CSV file
    data = data[:, 1:] # Remove the first column

    # return the data
    return data
```

The load_data function aims to load the S&P 500 stock data from a CSV file and convert it into a NumPy array that contains only the stock values. No arguments are required when calling this function since the file path and format are predetermined. Internally, it reads the CSV file, removes the first row (which contains company names) and the first column (which holds the dates), and returns the resulting numerical array. The output of load_data is essential as it supplies the raw data for further processing in functions like aggregate_data and calculate_mean_std.

```python
def aggregate_data( data ):
    # Aggregate the stock value of each company computing the LogRatio in blocks of 7 days (approx. a week).
    # Note that the stock markets close during weekends and bank holidays.


    num_days, num_companies = data.shape # Get the dimensions of the data

    num_weeks = num_days // 5 # Calculate the number of weeks in the data

    data_week = np.zeros((num_weeks, num_companies)) # Initialise the data_week array

    for week in range(num_weeks): # Loop through each week

        start_idx = week * 5 # Calculate the start index of the week
        end_idx = start_idx + 5 # Calculate the end index of the week

        if end_idx <= num_days: # Check if the end index is within the data
            log_ratio = np.log(data[end_idx-1, :] / data[start_idx, :]) # Calculate the log ratio of the week
            data_week[week, :] = log_ratio # Store the log ratio in the data_week array

    # Return the aggregated data called weekly_data
    return data_week
```

The aggregate_data function is designed to convert daily stock values into aggregated weekly data by computing the logarithmic ratio of stock prices between the first day of one week and the first day of the next. It requires a single argument, the data array produced by load_data, which contains the daily stock values. The function works by first creating an empty array sized to hold a row for each working week (calculated by dividing the total number of rows by the number of working days) and a column for each company. It then iterates over the dataset by determining a starting index for each week and calculating an ending index by adding 5 to the starting index. If the ending index is within the bounds of the data, it computes the logarithmic ratio between the stock values at these indices and stores that value in the weekly aggregated array. This aggregated data is used in later functions for objective evaluations and optimizations.

```python
def calculate_mean_std( data ):
    # Compute the aggregated data mean and the standard deviation for each company
    # Store them in a matrix called mean_std containing column-wise.
    # Column 0 should contain all companies' means,
    # and column 1 its corresponding standard deviation

    means = np.mean(data, axis=0) # Calculate the mean of the data
    stdv = np.std(data, axis=0) # Calculate the standard deviation of the data
    data_mean_std = np.column_stack((means, stdv)) # Stack the means and standard deviations column-wise

    # Return this matrix mean_std
    return data_mean_std
```

The calculate_mean_std function aims to compute the mean and the standard deviation for each company's stock values in the dataset, providing the statistical basis for subsequent simulations and evaluations. It takes the data array as its single input, which can be either daily or aggregated data. The function works by calculating the mean of each column (each column representing a company) and then computing the standard deviation for that same column. It combines these two arrays side by side to form a two-column matrix where the first column contains the means and the second contains the standard deviations. This matrix, often referred to as mean_std, is then returned and is used as an important argument in several objective function evaluations (such as objective_function_VaR, objective_function_sharp, and objective_function_mdd).

```python
def initial_solution( size, amount = 100 ):
    # Compute the initial solution
    # Giving our capital stored in variable amount (for instance, 100 Meuros)
    # generate the initial asset allocation assigning all money to a random company

    # return the initial solution
    isolution = np.zeros(size) # Initialise the initial solution
    random_company = np.random.randint(0, size) # Generate a random company
    isolution[random_company] = amount # Assign all money to the random company

    return isolution # Return the initial solution
```

The initial_solution function is created to generate an initial portfolio allocation for a given number of assets by investing all the available capital into one randomly selected asset. It requires two arguments: size, which specifies the number of assets, and an optional amount parameter, which defaults to 100, representing the total capital. The function works by generating an array of zeros of length equal to the number of assets, and then it randomly selects one index corresponding to a company where the full capital is allocated, leaving other entries at zero. The output of initial_solution is used as the starting allocation for optimization algorithms like simulated_annealing_algorithm and tabu_search_algorithm.

```python
def objective_function_VaR( solution, mean_std ):
    # We implement the Monte Carlo method to compute the VaR
    # Giving the mean and standard deviation of each company computed over the aggregated data
    # and the current solution, compute:
    # 1. For each company, draw a random value following a normal distribution with its mean and standard deviation.
    # 2. Use these random values and the current assets allocated to each company to compute the total gain
    # 3. Repeat 100 times points 1 and 2.
    # Compute the 95% one-tailed Value at Risk over the 100 total gains

    num_rep = 100  # Number of repetitions

    random_returns = np.random.normal(loc=mean_std[:, 0], scale=mean_std[:, 1], size=(num_rep, len(solution))) # Draw random values from the normal distribution
    total_gains = random_returns.dot(solution) # Compute the total gain
    VaR_95 = np.percentile(total_gains, 5) # Compute the 95% one-tailed Value at Risk

    # Return the VaR
    return sum(solution) + VaR_95
```

The objective_function_VaR function is meant to evaluate a portfolio allocation by estimating its Value at

Risk (VaR) using a Monte Carlo simulation approach. It requires two arguments: the solution array representing the current portfolio allocation and the mean_std matrix, which contains the historical mean returns and standard deviations produced by calculate_mean_std. Internally, it generates 100 sets of simulated returns for each asset using their respective normal distribution parameters. For each simulation, the function calculates the total portfolio gain or loss by combining these simulated returns with the allocation weights. It then determines the 5th percentile of the gain distribution to represent the worst expected performance at a 95% confidence level and adds this value to the total invested capital, returning the resulting VaR as a measure of risk.

```python
def objective_function_sharp( solution, mean_std ):
    # Giving the mean and standard deviation of each company computed over the aggregated data
    # and the current solution compute:
    # 1. For each company, draw a random value following a normal distribution with its mean and standard deviation.
    # 2. Use these random values and the current assets allocated to each company to compute the total gain
    # 3. Repeat 100 times points 1 and 2.
    # Compute the (adapted) Sharp Ratio over the 100 total gains
    # 1. Compute the mean of the gains
    # 2. Compute the standard deviation of the gains
    # 3. Apply the adapted formula for the Sharp Ratio
    # Return the sharp ratio

    num_rep = 100 # Number of repetitions
    random_returns = np.random.normal(loc=mean_std[:, 0], scale=mean_std[:, 1], size=(num_rep, len(solution))) # Draw random values from the normal distribution
    gain_returns = np.dot(random_returns, solution) # Compute the total gain
    mean_of_gains = np.mean(gain_returns) # Compute the mean of the gains
    stdev = np.std(gain_returns) # Compute the standard deviation of the gains

    sharp = mean_of_gains / stdev # Compute the Sharp Ratio
    return sharp # Return the Sharp Ratio
```

The objective_function_sharp function is used to evaluate a portfolio's performance based on a risk-adjusted return measure by computing the Sharpe Ratio. This function takes as arguments the portfolio allocation (solution) and the mean_std matrix derived from calculate_mean_std. It works by performing 100 simulations of portfolio returns; for each asset, the function generates random returns using a normal distribution defined by its mean and standard deviation. The simulated returns across the assets are used to calculate the overall portfolio gain for each iteration. After accumulating 100 such simulated gains, the function computes the mean and standard deviation of these gains and then calculates the Sharpe Ratio by taking the ratio of the mean return to the standard deviation. The resulting Sharpe Ratio serves as the objective value for performance evaluation.

```python
def objective_function_mdd( solution, mean_std ):
    # Giving the mean and standard deviation of each company computed over the aggregated data
    # and the current solution, compute:
    # 1. For each company, draw a random value following a normal distribution with its mean and standard deviation.
    # 2. Use these random values and the current assets allocated to each company to compute the total gain
    # 3. Repeat 100 times points 1 and 2.
    # Compute the (adapted) maximum drawdow (mdd) over the 100 total gains
    # 1. Find the minimum gain
    # 2. Find the maximum gain
    # 3. Apply the adapted formula for mdd
    # Return the mdd

    num_rep = 100 # Number of repetitions
    random_returns = np.random.normal(loc=mean_std[:, 0], scale=mean_std[:, 1], size=(num_rep, len(solution))) # Draw random values from the normal distribution
    gain_returns = np.dot(random_returns, solution) # Compute the total gain
    mdd = (np.min(gain_returns) - np.max(gain_returns)) / np.max(gain_returns) # Compute the maximum drawdown

    return mdd # Return the maximum drawdown
```

The objective_function_mdd function aims to evaluate the portfolio allocation in terms of risk by calculating the Maximum Drawdown (MDD), which measures the largest drop from a peak to a trough over the simulated outcomes. It requires the same two arguments as the previous objective functions, a portfolio allocation (solution) and the mean_std matrix containing the assets' statistical data. Within the function, simulated returns for each asset are generated using their corresponding means and standard deviations. These simulations are combined with the portfolio weights to produce overall portfolio gains for each simulation. The function then determines the difference between the highest and lowest of these simulated gains, which represents the worst-to-best performance swing, returning this difference as the MDD objective value used in the risk assessment of the portfolio.

```python
def get_neighbour(solution):

    neighbour = solution.copy() # Copy the solution

    companies_with_capital = np.where(neighbour > 0)[0] # Get the indices of the companies with capital

    if len(companies_with_capital) == 0: # If there are no companies with capital
        return neighbour # Return the solution

    source_company = np.random.choice(companies_with_capital) # Choose a random company with capital
    all_companies = np.arange(len(neighbour)) # Get all the companies
    destination_company = np.random.choice(all_companies) # Choose a random company

    amount_to_move = np.random.uniform(0.05, 0.1) * neighbour[source_company] # Calculate the amount to move

    neighbour[source_company] -= amount_to_move # Subtract the amount to move from the source company
    neighbour[destination_company] += amount_to_move # Add the amount to move to the destination company

    return neighbour # Return the neighbour
```

The get_neighbour function is designed to create a new candidate portfolio allocation, known as a "neighbor," by making a slight modification to the current solution. It requires a single argument: the current portfolio allocation (solution). The function operates by creating a copy of the existing solution to avoid modifying the original allocation. It then identifies which companies have a non-zero allocation, and if such companies exist, it randomly chooses one as the source from which to transfer a random fraction of its capital. It also randomly selects a destination company (which may have zero or non-zero allocation) to receive the transferred capital. The resulting neighbor solution, which slightly adjusts the capital distribution, is returned and is later used by both simulated and tabu search algorithms for exploration.

```python
def simulated_annealing_algorithm( data, mean_std, solution, temperature, alpha, num_iter, objective_function ):
    current_solution = solution # Set the current solution
    current_eval = objective_function(current_solution, mean_std) # Evaluate the current solution
    best = current_solution.copy() # Copy the current solution
    best_eval = current_eval # Set the best evaluation

    for _ in range(num_iter): # Loop through the iterations
        neighbour_solution = get_neighbour(current_solution) # Get the neighbour solution
        neighbour_eval = objective_function(neighbour_solution, mean_std) # Evaluate the neighbour solution

        delta = neighbour_eval - current_eval # Calculate the difference between the neighbour and current evaluation

        # Add a safety check to prevent overflow in the exponential calculation
        if delta > 0 or (temperature > 1e-10 and np.random.rand() < np.exp(min(-delta / temperature, 700))):
            current_solution = neighbour_solution # Set the current solution to the neighbour solution
            current_eval = neighbour_eval # Set the current evaluation to the neighbour evaluation

        if current_eval > best_eval: # If the current evaluation is better than the best evaluation
            best = current_solution.copy() # Copy the current solution
            best_eval = current_eval # Set the best evaluation to the current evaluation

        temperature *= alpha # Cool the temperature

    return best, best_eval # Return the best solution and its evaluation
```

The simulated_annealing_algorithm function is intended to optimize the portfolio allocation using the simulated annealing method. It requires several arguments: data (the aggregated market data from aggregate_data), mean_std (the statistical matrix from calculate_mean_std), an initial portfolio allocation (solution from initial_solution), a starting temperature (temperature) to control the acceptance probability of worse solutions, a cooling factor (alpha), the number of iterations (num_iter) to run the algorithm, and the objective_function that will evaluate portfolio performance (which could be any one of the objective functions such as objective_function_VaR, objective_function_sharp, or objective_function_mdd). The function works by starting with the initial solution and then iteratively generating a neighboring solution using get_neighbour. For each neighbor, it evaluates the performance change using the provided objective_function. If the new solution improves on the current performance,

or if it passes a probabilistic acceptance test (which becomes stricter as the temperature decreases due to the cooling factor alpha), it replaces the current solution. This iterative process continues until the specified number of iterations is reached, culminating in the best-found portfolio allocation that is returned as the optimal solution. The function is closely related to get_neighbour and leverages the objective functions for performance evaluation.

```
def tabu_list_management(tabu_list, solution, current_iteration, current_tabu_tenure):
    solution_key = tuple(solution.round(decimals=4)) # Round the solution to 4 decimal places and convert to a tuple
    tabu_list[solution_key] = current_iteration + current_tabu_tenure # Update the tabu list
    tabu_list = {sol: exp for sol, exp in tabu_list.items() if exp > current_iteration} # Remove entries that have expired
    return tabu_list # Return the updated tabu list
```

The tabu_list_management function is designed to manage a tabu list used in the tabu search algorithm to avoid cycling back to recently visited solutions. It requires four arguments: the current tabu list (tabu_list), the current portfolio allocation (solution), the current iteration number (current_iteration), and the current tabu tenure (current_tabu_tenure) which indicates how many iterations a solution should remain tabu. The function works by first rounding the solution values to four decimal places and converting the resulting array into a tuple so that it can be used as a key in the tabu list dictionary. It then inserts this key into the dictionary with an expiration iteration calculated by adding the current tabu tenure to the current iteration. After adding the new entry, the function goes through the existing entries in the tabu list and removes any whose expiration iterations have passed. Finally, it returns the updated tabu list, which is then used by the tabu_search_algorithm to check and enforce tabu restrictions during the search.

```
def evaluate_neighbours(neighbours, objective_function, mean_std):
    best_neig_eval = float('-inf') # Initialise the best neighbour evaluation
    best = None # Initialise the best neighbour

    for nei in neighbours: # Loop through the neighbours
        current_eval = objective_function(nei, mean_std) # Evaluate the neighbour using the objective function
        if current_eval > best_neig_eval: # If the current evaluation is better than the best neighbour evaluation
            best_neig_eval = current_eval # Set the best neighbour evaluation to the current evaluation
            best = nei # Set the best neighbour to the current neighbour

    return best # Return the best neighbour
```

The evaluate_neighbours function is used to compare a set of candidate neighbor solutions in order to select the best one based on a chosen performance metric. It takes three arguments: a list of neighbor solutions (neighbours), the objective_function to use for performance evaluation, and the mean_std matrix containing the historical statistics for each asset. The function works by iterating over each neighbor in the list, computing its performance using the objective_function in combination with the mean_std values. As it evaluates each candidate, it keeps track of the best performance score and updates the current best candidate whenever a better score is encountered. The final output is the neighbor with the highest performance, which can then be used to update the current solution in optimization algorithms. This function naturally ties into the optimization process and is called within both the simulated and tabu search algorithms to assess multiple candidate solutions.

```python
def tabu_search_algorithm( data, mean_std, solution, tabu_tenure, num_iter, objective_function, num_neighbours = 100, aspiration_threshold = 0.05)
    tabu_list = {} # Initialise the tabu list
    best = solution.copy() # Copy the initial solution
    best_eval = objective_function(solution, mean_std) # Evaluate the initial solution using the objective function

    current_tabu_tenure = tabu_tenure # Set the current tabu tenure
    iterations_without_improvement = 0 # Initialise the number of iterations without improvement

    for (variable) neighbours: list through the iterations
        neighbours = [] # Initialise the neighbours list
        neighbours_eval = [] # Initialise the neighbours evaluation list

        for j in range(num_neighbours): # Loop through the neighbours
            neighbour = get_neighbour(best) # Get the neighbour
            candidate_key = tuple(neighbour.round(decimals=4)) # Round the neighbour to 4 decimal places and convert to a tuple
            candidate_eval = objective_function(neighbour, mean_std) # Evaluate the neighbour using the objective function

            if candidate_key in tabu_list: # If the neighbour is in the tabu list
                if candidate_eval > best_eval * (1 + aspiration_threshold): # If the neighbour is better than the aspiration threshold
                    neighbours.append(neighbour) # Add the neighbour to the neighbours list
                    neighbours_eval.append(candidate_eval) # Add the neighbour evaluation to the neighbours evaluation list
            else: # If the neighbour is not in the tabu list
                neighbours.append(neighbour) # Add the neighbour to the neighbours list
                neighbours_eval.append(candidate_eval) # Add the neighbour evaluation to the neighbours evaluation list

        if not neighbours: # If there are no neighbours
            solution = initial_solution(len(best)) # Perform a random restart to allow for diversification
            current_eval = objective_function(solution, mean_std) # Evaluate the new solution using the objective function
        else: # If there are neighbours
            best_idx = np.argmax(neighbours_eval) # Get the index of the best neighbour
            solution = neighbours[best_idx] # Set the solution to the best neighbour
            current_eval = neighbours_eval[best_idx] # Set the current evaluation to the evaluation of the best neighbour

        if current_eval > best_eval: # If the current evaluation is better than the best evaluation
            best = solution.copy() # Copy the solution
            best_eval = current_eval # Set the best evaluation to the current evaluation
            iterations_without_improvement = 0 # Reset the number of iterations without improvement
        else: # If the current evaluation is not better than the best evaluation
            iterations_without_improvement += 1 # Increment the number of iterations without improvement

        tabu_list = tabu_list_management(tabu_list, solution, i, current_tabu_tenure) # Update the tabu list

        if iterations_without_improvement >= 10: # If the number of iterations without improvement is greater than 10
            current_tabu_tenure = min(current_tabu_tenure + 1, 20) # Increment the tabu tenure
        else: # If the number of iterations without improvement is less than 10
            current_tabu_tenure = tabu_tenure # Reset the tabu tenure


    # Return the best solution and its evaluation
    return best, best_eval
```

The tabu_search_algorithm function is designed to explore the space of portfolio allocations while avoiding recent, already-explored solutions by using a tabu list mechanism. It requires several arguments: data (the aggregated market data from aggregate_data), mean_std (the statistical matrix from calculate_mean_std), an initial portfolio allocation (solution, typically generated by initial_solution), the tabu_tenure which defines how long a solution should remain in the tabu list, the total number of iterations (num_iter) to run the search, and the objective_function to evaluate portfolio performance. The function works by initializing tracking variables such as the current best solution and its evaluation, and it creates an empty or prepopulated tabu list to store visited solutions. Within a loop, it generates neighbor solutions (using functions like get_neighbour) and evaluates them while checking against the tabu list or an aspiration criterion to override the tabu condition. If a neighbor not in the tabu list (or meeting the aspiration criteria) provides an improvement, the function updates the best solution and resets the counter for iterations without improvement. The tabu list is updated in each iteration by calling tabu_list_management, ensuring that outdated entries are removed. Once the number of iterations is completed, the function returns the best solution found, its evaluation, and the iteration number at which it occurred. This function is related to evaluate_neighbours and get_neighbour, as they form part of the candidate solution generation and evaluation process within the tabu search framework.

Class OptimisationResults(abc,ABC)

The objective of the OptimisationResults class is to encapsulate and manage all the outcome information from an optimization process in a structured way. This class is intended to store details such as the best portfolio allocation obtained, the corresponding evaluation of that allocation according to the chosen objective function, and additional metadata like the number of iterations or the point in the search where

the optimum was achieved. In our context, after running either the simulated annealing algorithm or the tabu search algorithm, an instance of OptimisationResults collects and organizes these results, making it easier to report, compare, and analyze the performance of the optimization over different runs or varying parameters. Essentially, the class acts as a container that holds all the critical output information from the optimization procedures, thereby promoting modularity and clarity when handling and interpreting the results.

```python
def __init__(self, columns=[], objective_functions=[], path='', amount=100, repetitions_per_run=10, seed=0): # Initialise the class
    """
    This is the base class for the simulated annealing and tabu search algorithms.
    It is used to store the results of the experiments and to save them to an Excel file.
    args:
        columns: list of column names for the different configurations in the output Excel
        objective_functions: list of objective function references to test
        path: path to the Excel file for saving results
        amount: initial capital amount
        repetitions_per_run: number of repetitions per run
        seed: seed for the random number generator

    returns:
        None
    """

    self.amount = amount # Initial capital amount
    self.repetitions_per_run = max(1, repetitions_per_run) # Number of repetitions per run
    self.columns = columns # List of column names for the different configurations in the output Excel
    self.objective_functions = objective_functions # List of objective function references to test
    self.path = path # Path to the Excel file for saving results

    if seed is not None: # If a seed is provided
        np.random.seed(seed) # Set the seed

    self.data = load_data() # Load the data
    self.data_week = aggregate_data(self.data) # Aggregate the data
    self.data_mean_std = calculate_mean_std(self.data_week) # Calculate the mean and standard deviation of the data
    self.isolution = initial_solution(self.data_week.shape[1], self.amount) # Initialise the solution

    self.initial_metrics = {} # Initialise the initial metrics
    for obj_func in self.objective_functions: # Loop through the objective functions
        self.initial_metrics[obj_func.__name__] = obj_func(self.isolution.copy(), self.data_mean_std) # Evaluate the initial solution

    self.metric_row_map = { # Map the objective functions to the rows in the Excel
        'objective_function_VaR': 'VaR at 95%',
        'objective_function_sharp': 'Sharp',
        'objective_function_mdd': 'MDD'
    }
    self.df = None # Initialise the DataFrame
```

Initialize the experiment's common settings and load initial data. It sets up the parameters used across experiments and computes the initial metrics using the supplied objective functions. It sets instance variables with the provided values. If a seed is specified, it sets NumPy's random seed for consistency. Loads data via helper functions (load_data, aggregate_data, and calculate_mean_std) and creates an initial solution using a helper (initial_solution); these functions work together to prepare the data and initial state for the optimisation. Computes and stores initial metrics for each objective function by evaluating the initial solution. Defines a mapping (metric_row_map) that links objective function names to descriptive row labels for the Excel output.Initializes self.df (the DataFrame for results) to later be populated or created.

```python
def load_or_create_excel(self):
    """
    Loads existing results from an Excel file or creates a new DataFrame if the file doesn't exist.
    Ensures all expected rows and columns are present in the DataFrame.

    returns:
        pandas.DataFrame: DataFrame with optimisation results
    """
    try:
        self.df = pd.read_excel(self.path, index_col=0)
        print(f"Loaded existing results from {self.path}")
    except FileNotFoundError:
        print(f"Creating new results DataFrame for {self.path}")
        rows = [self.metric_row_map[obj_func.__name__] for obj_func in self.objective_functions] + ['Execution Time'] # Create the rows
        initial_cols = ['Initial Values'] + self.columns # Create the columns
        self.df = pd.DataFrame(index=rows, columns=initial_cols) # Create the DataFrame
        self.df = self.df.fillna(pd.NA) # Fill the missing values with NA

    expected_cols = ['Initial Values'] + self.columns # Create the expected columns
    for col in expected_cols: # Loop through the expected columns
        if col not in self.df.columns: # If the column does not exist
            self.df[col] = pd.NA # Fill the missing values with NA

    expected_rows = [self.metric_row_map[obj_func.__name__] for obj_func in self.objective_functions] + ['Execution Time'] # Create the expected rows
    for row in expected_rows: # Loop through the expected rows
        if row not in self.df.index: # If the row does not exist
            self.df.loc[row] = pd.NA # Fill the missing values with NA
                (variable) initial_value: Any
    for obj_func, initial_value in self.initial_metrics.items(): # Loop through the initial metrics
        row_name = self.metric_row_map.get(obj_func, None) # Get the row name
        if row_name: # If the row name exists
            self.df.loc[row_name, 'Initial Values'] = initial_value # Fill the initial values
    self.df.loc['Execution Time', 'Initial Values'] = '-' # Fill the execution time with '-'

    print("Initial DataFrame state:")
    print(self.df)
    return self.df # Return the DataFrame
```

The goal of this function is to ensure that there is a consistent DataFrame available for storing experiment results by loading an existing Excel file or creating a new one if it doesn't exist. It uses instance variables (such as self.path, self.columns, and self.objective_functions) as its arguments and works by attempting to read an Excel file with pd.read_excel; on FileNotFoundError, it constructs a new DataFrame with rows derived from the metric mapping (plus an "Execution Time" row) and columns beginning with "Initial Values" followed by configuration names. The method then populates missing rows or columns with default NA values and updates the "Initial Values" column with the precomputed metrics, ensuring that the DataFrame is fully prepared for subsequent result insertion.

```python
def save_results(self, avg_evaluations, avg_times, config_index):
    """
    Saves the average evaluation results and execution times to the DataFrame.

    args:
        avg_evaluations: dictionary of average evaluation results for each objective function
        avg_times: dictionary or float with average execution times
        config_index: index of the configuration in the columns list

    returns:
        pandas.DataFrame: Updated DataFrame with new results
    """
    if config_index >= len(self.columns): # If the configuration index is out of bounds
        print(f"Warning: Configuration index {config_index} out of bounds for columns {self.columns}. Skipping save.") # Print a warning
        return self.df # Return the DataFrame

    col_name = self.columns[config_index] # Get the column name

    for obj_func, avg_eval in avg_evaluations.items(): # Loop through the average evaluations
        row_name = self.metric_row_map.get(obj_func.__name__, None) # Get the row name
        if row_name: # If the row name exists
            if isinstance(avg_eval, float): # If the average evaluation is a float
                self.df.loc[row_name, col_name] = f"{avg_eval:.2f}" # Fill the average evaluation
            else: # If the average evaluation is not a float
                self.df.loc[row_name, col_name] = avg_eval


    if isinstance(avg_times, dict): # If the average times is a dictionary
        avg_exec_time = sum(avg_times.values()) / len(avg_times) if avg_times else 0 # Calculate the average execution time
    else: # If the average times is not a dictionary
        avg_exec_time = avg_times # Set the average execution time to the average times

    self.df.loc['Execution Time', col_name] = f"{avg_exec_time:.3f}s" # Fill the execution time
    return self.df # Return the DataFrame
```

The purpose of save_results is to record and update the DataFrame with computed average evaluation

results and execution times for a specific configuration. Accepting avg_evaluations (a dictionary mapping objective functions to their average scores), avg_times (the average execution time or a dictionary thereof), and config_index (which identifies the target configuration), it first validates the provided index. It then iterates over the objective functions, retrieves corresponding row labels using the mapping, and updates each cell with formatted average values, while also computing a consolidated average execution time and updating the "Execution Time" row in the DataFrame. This method ties into perform_experiments by finalising the result storage for each configuration after multiple experiment repetitions.

```
@staticmethod
def _exec_time(function, *args, **kwargs):
    """
    Calculates the execution time of a function.

    args:
        function: the function to execute
        *args, **kwargs: arguments to pass to the function

    returns:
        tuple: (function_result, execution_time)
    """
    start_time = time.perf_counter() # Start the timer
    result = function(*args, **kwargs) # Execute the function
    exec_time = time.perf_counter() - start_time # Calculate the execution time
    return result, exec_time # Return the result and the execution time
```

The goal of the static method _exec_time is to provide a timing utility that measures the execution time of any supplied function. It takes as arguments a function and an arbitrary collection of positional and keyword arguments, records the start time using a high-precision timer, executes the function with the provided arguments, and then computes the elapsed time. It returns a tuple containing the function's output and the measured execution time, enabling both performance benchmarking and integration within the optimisation experiment workflow as used by the _run_single_optimisation implementations.

```
@abc.abstractmethod
def _run_single_optimisation(self, config_index, objective_function):
    """
    Abstract method to run a single optimisation experiment.
    Must be implemented by subclasses.

    args:
        config_index: index of the configuration in the parameters lists
        objective_function: objective function to optimise

    returns:
        tuple: ((best_solution, best_evaluation), execution_time)
    """
    pass
```

This abstract method's purpose is to define a mandatory interface for running a single optimisation

experiment, requiring subclasses to specify how an experiment run is conducted. It accepts a config_index (to identify the relevant configuration parameters) and an objective_function (the evaluation function guiding the optimisation) and is expected to return a tuple containing the best solution, its evaluation, and the execution time. As an abstract placeholder, it ensures that subclasses like Results_Simulated_Annealing and Results_Tabu_Search implement their own specific versions that integrate with the common execution timing provided by _exec_time.

```python
def perform_experiments(self):
    """
    Performs all the optimisation experiments according to the configurations.
    For each configuration and objective function, runs multiple repetitions and
    calculates average evaluation results and execution times.
    Saves results to the Excel file after each configuration is completed.

    returns:
        pandas.DataFrame: Final DataFrame with all results
    """
    self.df = self.load_or_create_excel() # Load or create the Excel file
    num_configs = len(self.columns) # Get the number of configurations

    for i in range(num_configs): # Loop through the configurations
        print(f"\n--- Running Configuration: {self.columns[i]} ---") # Print the configuration
        avg_evaluations = {} # Initialise the average evaluations
        average_times = {} # Initialise the average times

        for obj_func in self.objective_functions: # Loop through the objective functions
            print(f"  Optimising for: {obj_func.__name__} ({self.repetitions_per_run} repetitions)") # Print the objective function
            eval_results_list = [] # Initialise the evaluation results list
            exec_times_list = [] # Initialise the execution times list

            for rep in range(self.repetitions_per_run): # Loop through the repetitions
                (best_sol, best_eval), exec_time = self._run_single_optimisation(i, obj_func) # Run the single optimisation
                eval_results_list.append(best_eval) # Append the evaluation results
                exec_times_list.append(exec_time) # Append the execution times
                print(f"    Rep {rep+1}/{self.repetitions_per_run}: Eval={best_eval:.4f}, Time={exec_time:.4f}s") # Print the evaluation and execution time

            if eval_results_list: # If the evaluation results list is not empty
                avg_eval = np.mean(eval_results_list) # Calculate the average evaluation
                average_time = np.mean(exec_times_list) # Calculate the average time
                avg_evaluations[obj_func] = avg_eval # Fill the average evaluations
                average_times[obj_func] = average_time # Fill the average times
                print(f"  -> Avg Eval: {avg_eval:.4f}, Avg Time: {average_time:.4f}s") # Print the average evaluation and time
            else: # If the evaluation results list is empty
                avg_evaluations[obj_func] = pd.NA # Fill the average evaluations with NA

        self.df = self.save_results(avg_evaluations, average_time, i) # Save the results
        try:
            self.df.to_excel(self.path) # Save the results to the Excel file
            print(f"Saved intermediate results to {self.path}")
        except Exception as e:
            print(f"Error saving Excel file: {e}") # Print an error message


    print("\n--- Experiment Complete ---")
    print("Final Results DataFrame:")
    print(self.df)
    return self.df # Return the DataFrame
```

The aim of perform_experiments is to orchestrate a complete experiment cycle by iterating over all configuration columns and objective functions, running multiple repetitions per configuration, averaging the collected evaluations and execution times, and updating the results DataFrame accordingly. It requires no external arguments and works by first acquiring a DataFrame through load_or_create_excel, then for each configuration it repeatedly calls the subclass-implemented _run_single_optimisation method, aggregates the results, and finally uses save_results to update and persist the DataFrame to an Excel file. This method integrates all previously defined functions into a complete experimental workflow, ensuring that each configuration's performance metrics are measured, averaged, and stored.

class Results_Simulated_Annealing(OptimisationResults):

This subclass of OptimisationResults is tailored for simulated annealing optimisation experiments, extending the base functionality with parameters specific to simulated annealing. Its goal is to incorporate settings like the number of iterations, initial temperatures, and cooling rates (alpha values) for each

configuration, while leveraging the common experiment framework for data handling and result storage inherited from the base class.

```python
def __init__(self, num_iters=[100], temp=[110], alpha=[0.95], **kwargs):
    """
    Subclass for running Simulated Annealing optimisation experiments.

    args:
        num_iters: list of number of iterations for each configuration
        temp: list of initial temperatures for each configuration
        alpha: list of cooling rates for each configuration
        **kwargs: arguments to pass to the parent class

    returns:
        None
    """
    super().__init__(**kwargs) # Initialise the parent class
    self.num_iters = num_iters # Fill the number of iterations
    self.temp = temp # Fill the temperature
    self.alpha = alpha # Fill the alpha

    n_cols = len(self.columns) # Get the number of columns
    if len(self.num_iters) != n_cols or len(self.temp) != n_cols or len(self.alpha) != n_cols: # If the lengths of the parameter lists do not match the number of columns
        print("Warning: Lengths of SA parameter lists (num_iters, temp, alpha) do not match the number of columns.") # Print a warning
```

The init method for the ResultsSimulatedAnnealing class is designed to create a new instance of the class that acts as a container for all critical results from a simulated annealing optimization run. When this constructor is called, it requires input values that are typically generated by the simulated_annealing_algorithm function, such as the best portfolio allocation found (best_solution), the corresponding objective evaluation (best_evaluation), the specific iteration at which this optimum was achieved (iteration_best), and possibly other related metrics like the execution time or final temperature. Internally, the init method assigns these provided values to the instance's attributes so that they can be easily accessed later for reporting, analysis, and comparisons with other optimization runs. By encapsulating the outcome of the simulated annealing process in a structured manner, this method helps maintain a clear separation between the optimization process and the result analysis, allowing other parts of the program to leverage these stored results in a consistent way.

```python
def _get_params(self, config_index): # Get the parameters

    iters = self.num_iters[config_index] if config_index < len(self.num_iters) else self.num_iters[-1] # Get the number of iterations
    temp_val = self.temp[config_index] if config_index < len(self.temp) else self.temp[-1] # Get the temperature
    alpha_val = self.alpha[config_index] if config_index < len(self.alpha) else self.alpha[-1] # Get the alpha
    return iters, temp_val, alpha_val # Return the number of iterations, temperature and alpha
```

The goal of _get_params is to retrieve the appropriate simulated annealing parameters for a given configuration index; it accepts config_index as its sole argument and works by indexing the parameter lists (num_iters, temp, alpha) to return a tuple of values. If the index exceeds the available list length, it defaults to the last element in each list, thereby ensuring that valid parameters are always supplied. This method centralises parameter extraction and directly feeds into the subsequent optimisation routine.

```python
def _run_single_optimisation(self, config_index, objective_function): # Run the single optimisation

    iters, temp_val, alpha_val = self._get_params(config_index) # Get the parameters

    solution_copy = self.isolution.copy() # Copy the solution

    result, exec_time = self._exec_time(
        simulated_annealing_algorithm, # Call the simulated annealing algorithm
        self.data_week,
        self.data_mean_std,
        solution_copy,
        temp_val,
        alpha_val,
        iters,
        objective_function
    )
    return result, exec_time # Return the result and the execution time
```

The purpose of this method is to execute a single optimisation run using the simulated annealing algorithm. Its arguments are config_index, which determines which set of parameters to use, and objective_function, which directs the evaluation. It first calls _get_params to obtain the relevant iterations, temperature, and alpha, then creates a copy of the initial solution to avoid in-place modifications. It proceeds to time the execution of the simulated_annealing_algorithm using the _exec_time helper method, passing the necessary data and SA-specific parameters; the method then returns the result and execution time, thus fulfilling the abstract interface defined in the base class for simulated annealing experiments.

Class Results_Tabu_Search:

This subclass of OptimisationResults is designed specifically for running tabu search optimisation experiments, adding parameters uniquely relevant to the tabu search algorithm (such as iterations, number of neighbours, tabu tenure, and an aspiration threshold) to the inherited framework. Its goal is to manage and execute tabu search experiments while reusing common functionality for data management, result recording, and performance measurement from its parent class.

```python
def __init__(self, num_iters=[100], num_neighbours=[50], tabu_tenure=[10], aspiration_threshold=[0.05], **kwargs):
    """
    Subclass for running Tabu Search optimisation experiments.

    args:
        num_iters: list of number of iterations for each configuration
        num_neighbours: list of number of neighbours to evaluate for each configuration
        tabu_tenure: list of tabu tenures for each configuration
        aspiration_threshold: list of aspiration thresholds for each configuration
        **kwargs: arguments to pass to the parent class

    returns:
        None
    """
    super().__init__(**kwargs) # Initialise the parent class
    self.num_iters = num_iters # Fill the number of iterations
    self.num_neighbours = num_neighbours # Fill the number of neighbours
    self.tabu_tenure = tabu_tenure # Fill the tabu tenure
    self.aspiration_threshold = aspiration_threshold # Fill the aspiration threshold

    n_cols = len(self.columns) # Get the number of columns
    if len(self.num_iters) != n_cols or len(self.num_neighbours) != n_cols or len(self.tabu_tenure) != n_cols: # If the lengths of the parameter lists do not match the number of columns
        print("Warning: Lengths of TS parameter lists (num_iters, num_neighbours, tabu_tenure) do not match the number of columns.") # Print a warning
```

The constructor for Results_Tabu_Search receives TS-specific parameter lists (num_iters, num_neighbours, tabu_tenure, aspiration_threshold) along with common keyword arguments passed to the OptimisationResults constructor. Its goal is to initialise both the inherited properties and additional tabu search parameters; it assigns these values to instance variables and verifies that the lengths of these parameter lists match the number of configuration columns, issuing warnings if there is a mismatch. This setup ensures that each configuration is provided with the correct tabu search settings for subsequent experiments.

```python
def _get_params(self, config_index): # Get the parameters

    iters = self.num_iters[config_index] if config_index < len(self.num_iters) else self.num_iters[-1] # Get the number of iterations
    neighbours = self.num_neighbours[config_index] if config_index < len(self.num_neighbours) else self.num_neighbours[-1] # Get the number of neighbours
    tenure = self.tabu_tenure[config_index] if config_index < len(self.tabu_tenure) else self.tabu_tenure[-1] # Get the tabu tenure
    threshold = self.aspiration_threshold[config_index] if config_index < len(self.aspiration_threshold) else self.aspiration_threshold[-1] # Get the aspiration threshold
    return iters, neighbours, tenure, threshold # Return the number of iterations, neighbours, tenure and aspiration threshold
```

The _get_params method in this subclass is tasked with obtaining the relevant tabu search parameters for a given configuration, taking config_index as its input. It works by accessing each TS-specific list (num_iters, num_neighbours, tabu_tenure, aspiration_threshold) at the given index, defaulting to the last element if the index is out of range, and returns a tuple containing these values. This method centralises parameter extraction for each experiment run and supports the main optimisation execution method.

```python
def _run_single_optimisation(self, config_index, objective_function): # Run the single optimisation

    iters, neighbours_val, tenure_val, threshold_val = self._get_params(config_index) # Get the parameters


    solution_copy = self.isolution.copy() # Copy the solution

    result, exec_time = self._exec_time(
        tabu_search_algorithm, # Call the tabu search algorithm
        self.data_week,
        self.data_mean_std,
        solution_copy,
        tenure_val,
        iters,
        objective_function,
        num_neighbours=neighbours_val,
        aspiration_threshold=threshold_val
    )
    return result, exec_time # Return the result and the execution time
```

The goal of _run_single_optimisation in this subclass is to run a single iteration of the tabu search algorithm and record its performance. It accepts config_index and objective_function as arguments, retrieves the appropriate tabu search parameters by calling _get_params, and creates a copy of the initial solution to prevent unwanted side effects. It then calls the tabu_search_algorithm function—wrapped with the _exec_time helper to measure execution time—passing the required data and TS-specific parameters. Finally, it returns the best solution along with its evaluation and the execution time, thereby satisfying the abstract method contract established in OptimisationResults and integrating seamlessly with the overall experiment framework.

MAIN SESSION 3

```python
# --- Main Execution Block Session 3 ---
if __name__ == "__main__":

    #Inside of the folder there is an empty template that is tailored to the needs of the simulated annealing and tabu search algorithms classes.
    #I will leave it empty if there is a need to run the simulations again.

    # Define common parameters
    output_excel_file = 'Results_improvements_refactored.xlsx'
    objective_functions_to_run = [objective_function_VaR, objective_function_sharp, objective_function_mdd]
    repetitions_for_average = 10

    # --- Simulated Annealing Experiments ---
    print("="*20 + " Running Simulated Annealing " + "="*20)
    sa_num_iters =      [100,   1000,   100,    1000,   1000] # Base, Impr-1, Impr-2, Impr-3, Impr-4
    sa_temp =           [110,   110,    140,    110,    150]
    sa_alpha =          [0.95,  0.95,   0.95,   0.99,   0.99]
    sa_columns = ['Simulated Annealing Base Case', 'Simulated Annealing Improv-1', 'Simulated Annealing Improv-2', 'Simulated Annealing Improv-3', 'Simulated Annealing Improv-4']

    sa_runner = Results_Simulated_Annealing(
        num_iters=sa_num_iters,
        temp=sa_temp,
        alpha=sa_alpha,
        columns=sa_columns,
        objective_functions=objective_functions_to_run,
        path=output_excel_file,
        repetitions_per_run=repetitions_for_average,
        seed=0
    )
    final_sa_df = sa_runner.perform_experiments()
    print("\nSimulated Annealing Final Results:")

    display_cols_sa = ['Initial Values'] + sa_columns
    print(final_sa_df[display_cols_sa])
    # --- Tabu Search Experiments ---
    print("\n" + "="*20 + " Running Tabu Search " + "="*20)
    ts_num_iters =      [100,  1000,   100,   100,   100] # Base, Impr-1, Impr-2, Impr-3, Impr-4
    ts_num_neighbours = [10,   10,    50,   10,   10]
    ts_tabu_tenure =    [10,   10,    10,    30,    10]
    ts_aspiration_threshold = [0.05, 0.05, 0.05, 0.05, 0.03]
    ts_columns = ['Tabu Search Base Case', 'Tabu Search Improv-1', 'Tabu Search Improv-2', 'Tabu Search Improv-3', 'Tabu Search Improv-4']

    ts_runner = Results_Tabu_Search(
        num_iters=ts_num_iters,
        num_neighbours=ts_num_neighbours,
        tabu_tenure=ts_tabu_tenure,
        aspiration_threshold=ts_aspiration_threshold,
        columns=ts_columns,
        objective_functions=objective_functions_to_run,
        path=output_excel_file,
        repetitions_per_run=repetitions_for_average,
        seed=0
    )
    final_ts_df = ts_runner.perform_experiments()
    print("\nTabu Search Final Results (including previous SA runs):")
    print(final_ts_df)
```

The main function from Session 3 acts as the central orchestrator for the entire portfolio optimization process under the improvements explored in this session. Its goal is to integrate all the components—data loading, data aggregation, statistical computation, initial solution generation, and the execution of both simulated annealing and tabu search algorithms—in order to compare and assess the performance of the different optimization approaches. When this main function is executed, it does not require any external arguments; instead, it internally calls functions such as load_data to retrieve the raw S&P 500 stock data, aggregate_data to compute the weekly log returns, and calculate_mean_std to generate the matrix of mean returns and standard deviations. The function then creates an initial portfolio allocation by invoking initial_solution and sets various parameter values like iteration counts, starting temperatures, cooling factors (alpha), and tabu tenure for the different optimization algorithms. After these initial setup steps, the main function executes the simulated_annealing_algorithm and tabu_search_algorithm—each of which relies on auxiliary functions such as get_neighbour and, in the case of tabu search, tabu_list_management—to explore the solution space based on objective functions like VaR, Sharpe Ratio, or Maximum Drawdown. Throughout this process, the function collects and compares the optimization results, encapsulating them in corresponding results classes (for example, ResultsSimulatedAnnealing) for further analysis and reporting. In essence, the main function in Session 3 combines the responsibilities of data preprocessing, parameter initialization, and algorithm execution in a seamless workflow that allows for the systematic evaluation and benchmarking of the optimization improvements made in this session.

2. Results obtained

Initial values

- **Data shape (3715, 336)**

It prints the shape of the initial data, indicating that the database has the stock values of 336 companies (columns) throughout 3715 days (rows).

- **Weekly data shape (743, 336)**

Then, the shape of the array with all the companies' weekly log ratio, indicating that it has still 336 companies (columns) throughout 743 weeks (rows).

- **Initial distribution array**

We can also see the array with the 336 companies and how is the initial solution distributed (to which company we allocate the initial 100).

- **VaR at 95% confidence level: $93.13**

This is the first value of VaR we get with investing the $100M into the randomly selected company from the initial solution. The VaR estimates how much a certain investment might lose in normal market conditions in a given period of time. With a 95% confidence level, it estimates the worst expected loss (since we are working with the weekly log ratios, it would be over a week) in the bottom 5% of cases under normal conditions. This result means that in the worst 5% of weeks, the investment could drop to $93.13M, in other words, a potential loss of $6.67M.

- **Sharp: -0.05**

This is the first value of the Sharpe ratio we get with investing $100M into the randomly selected company from the initial solution. The Sharpe ratio compares the expected return of an investment with its associated risk (volatility), that is the mean of the gains divided by its standard deviation. A -0.05 Sharpe ratio indicates a small negative expected return relative to its risk, that is the investment is likely to lose money on average, and as the decimal is low it has a lot of volatility, ending in a bad combination of low return and high risk.

- **MDD: -1.71**

This is the first value of the MDD we get with investing $100M into the randomly selected company from the initial solution. The MDD measures the largest drop from a peak to a bottom in an investment value, showing how much you could lose in the worst case scenario. A -1.71 means that in the worst case scenario the investment could decrease by 171% from the highest to the lowest point in the simulations, in other words, it's a very unstable and risky investment.

Simulated Annealing Results:

In order to improve the asset allocation, Simulated Annealing was used by optimizing it to the three objective functions (VaR, Sharpe and MDD). The parameters used in this execution are 100 iterations, an initial temperature of 110 (so it allows bad moves early, exploration) and a cooling rate of 0.95 (so it gradually focuses the search on better solutions, exploitation, as the algorithm progresses.

- **Best VaR Evaluation: 96.0936730196923**

The best VaR found is around $96.09M, which means that in the worst 5% of simulated weeks, the portfolio could drop to $96.09M, in other words, a potential loss of $3.91M. This is an improvement over the initial value since a higher VaR means a lower risk of loss.

- **Best Sharpe Evaluation: 0.3288401269108972**

The best Sharpe ratio found is around 0.33, which means that the portfolio has a positive expected return compared to its risk, and also it is a more stable investment. This is an improvement over the initial value since it's positive (there is expected return) and a higher decimal (less volatility).

- **Best MDD Evaluation: -1.490259841840612**

The best MDD found is around -1.49, meaning that in the worst case the portfolio could drop by around 149% from the highest to the lowest point. Although it's still large, it is an improvement from the initial solution, so there are less extreme losses.

<u>Tabu Search Results:</u>

In order to improve the asset allocation, Tabu Search was used by optimizing it to the three objective functions (VaR, Sharpe and MDD). The parameters used in this execution are 100 iterations, 10 neighbours per iteration (increasing exploration), a tabu tenure of 10 (so it doesn't cycle and get stuck on local optima) and an aspiration threshold of 0.05 (allowing good solutions to pass the tabu restriction, balancing exploration and exploitation).

- **Best VaR Evaluation: 97.17700266877033**

The best VaR found is around $97.18M, which means that in the worst 5% of simulated weeks, the portfolio could drop to $97.18M, in other words, a potential loss of $2.88M. This is an improvement over the initial value since a higher VaR means a lower risk of loss. It also improves the result using simulated annealing, showing that this result has a lowest risk of loss.

- **Best Sharpe Evaluation: 0.42232065782766165**

The best Sharpe ratio found is around 0.42, which means that the portfolio has a positive expected return compared to its risk, and also it is a more stable investment. This is an improvement over the initial value since it's positive (there is expected return) and a higher decimal (less volatility). It also improves the result using simulated annealing, showing also a positive expected return and as the value increased, even less volatility.

- **Best MDD Evaluation: -1.4016570475456074**

The best MDD found is around -1.40, meaning that in the worst case the portfolio could drop by around 140% from the highest to the lowest point. Although it's still large, it is an improvement from the initial solution and the simulated annealing solution, so there are less extreme losses.

Simulated Annealing Final Results:

	Initial Values	Base Case	Improv-1	Imrpov-2	Improv-3	Improv-4
VaR at 95%	93.125283	96.57	97.38	96.74	97.81	97.61
Sharpe	-0.047059	0.28	0.34	0.28	0.38	0.37
MDD	-1.706414	-1.54	-1.46	-1.54	-1.45	-1.43
Execution time	-	0.109 s	1.066 s	0.100 s	1.078 s	1.134 s

The initial values are the ones calculated using the randomly generated initial solution, where the $100M are invested in a single random company.

The SA base case optimises the asset allocation with the initial parameters, and since the initial solution is randomly generated again, the numbers are not identical as before, even though the algorithm functions the same way. These value will serve as a reference point for the improvements.

- **Improvement 1:**

For the first improvement, we increase the number of iterations to 1000. This allows more time for the algorithm to explore and refine the solution, which improves exploitation with a deeper search. There are significant improvements in the VaR and the MDD, and a slight one in the Sharpe. However, the execution time is much higher given the high number of iterations.

- **Improvement 2:**

For the second improvement, we increase the initial temperature to 140. This boosts the exploration in the early phase by allowing more bad solutions to be accepted at the start. There is a slight improvement in the VaR while the Sharpe and MDD stay the same. Moreover, the execution time is similar, with a minor decrease.

- **Improvement 3:**

For the third improvement, we increase the number of iterations to 1000 and the cooling rate to 0.99. This allows more time for the algorithm to explore and refine the solution, and the temperature will decrease very slowly allowing it to stay in the exploration phase for longer. There are significant improvements in the VaR , Sharpe and MDD. However, the execution time is much higher given the high number of iterations.

- **Improvement 4:**

For the fourth improvement, we increase the number of iterations to 1000, the temperature to 150 and the cooling rate to 0.99. This allows more time for the algorithm to explore and refine the solution, and it increases the chance of accepting worse solutions at the beginning boosting early exploration (even more than with improvement 3), and with the temperature dropping slowly it prolongates this exploration phase. There are significant improvements in the VaR , Sharpe and MDD, although improvement 3 is slightly better. However, the execution time is much higher given the high number of iterations.

Tabu Search Final Results:

	Initial Values	Base Case	Improv-1	Imrpov-2	Improv-3	Improv-4
VaR at 95%	93.125283	96.83	97.23	96.93	96.55	96.79
Sharpe	-0.047059	0.37	0.43	0.40	0.34	0.39
MDD	-1.706414	-1.44	-1.40	-1.3	-1.44	-1.46
Execution time	-	0.103 s	11.041 s	5.443 s	1.109 s	1.100 s

The initial values are the ones calculated using the randomly generated initial solution, where the $100M are invested in a single random company.

The Tabu Search base case optimises the asset allocation with the initial parameters, and since the initial solution is randomly generated again, the numbers are not identical as before, even though the algorithm functions the same way. These value will serve as a reference point for the improvements.

- **Improvement 1:**

For the first improvement, we increase the number of iterations to 1000. This allows more time for the algorithm to explore and refine the solution, which improves exploitation with a deeper search. There is a significant improvements in the VaR and slight ones in the Sharpe and MDD. However, the execution time is much higher given the high number of iterations.

- **Improvement 2:**

For the second improvement, we increase the number of neighbours to 50. This will make the search more broad in every iteration, boosting exploration by considering more possible reallocations. There is a significant improvement in the MDD and slight ones in the VaR and Sharpe. However, the execution time is much higher given it evaluates more neighbours per iteration.

- **Improvement 3:**

For the third improvement, we increase the tabu tenure to 30. This strengthens the memory by blocking recently visited solutions for longer, boosting exploitation. There is a slight improvement in the Sharpe, the MDD stays the same and the VaR worsens. Moreover, the execution time is slightly higher given the management of a longer tabu list.

- **Improvement 4:**

For the fourth improvement, we decrease the aspiration threshold to 0.03. This allows stricter exploitation as it's harder for tabu moves to be accepted, even if slightly better. There is a slight improvement in the Sharpe and the Var and MDD worsen. Moreover, the execution time is slightly higher given the stricter acceptance criteria.

3. Group management

The communication in the group has been clear and the tasks were divided equally, given each member's strengths. There were no major issues related to group management, as the group management was effective.

4. Contributions of each member

Alan worked mostly on the code, although all the group shared insights on it, and Mireia helped clear doubts that raised in the code and concepts. Gerard, Natan and Mireia mostly worked on the detailed explanation of the document, providing explanations for the code and the results.

5. Self-evaluation

	Evaluated			
Evaluator	Gerard	Natan	Alan	Mireia
Gerard	-	10/10	10/10	10/10
Natan	10/10	-	10/10	10/10
Alan	10/10	10/10	-	10/10
Mireia	10/10	10/10	10/10	-